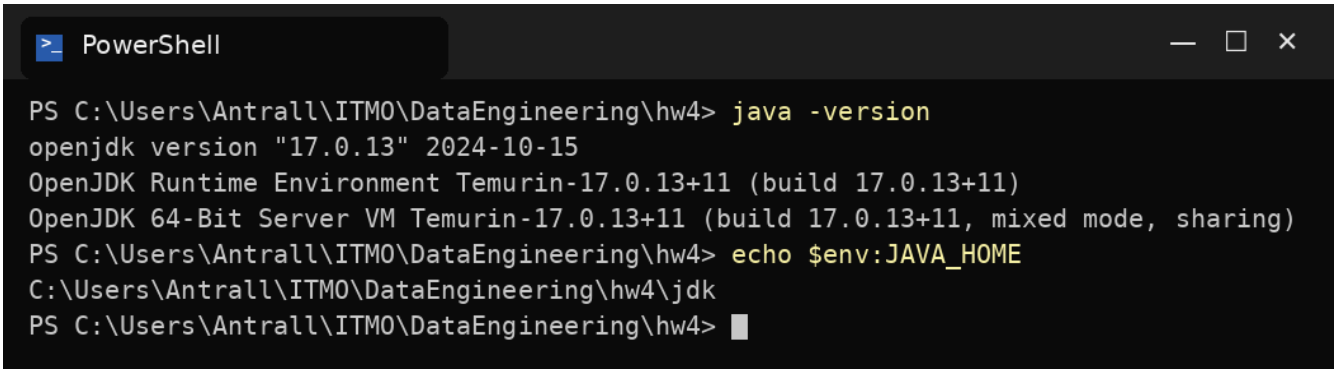
Домашнее задание №4: Отказоустойчивость на Apache ZooKeeper

Коханенко Вадим Вячеславович, группа J3310. 31.10.2026

1. Установка и подготовка

Поставил Java 17 (Temurin 17.0.13) в папку `jdk/` и задал `JAVA_HOME` на неё, скачал ZooKeeper 3.9.6 (контрольная сумма SHA-512 архива совпала с опубликованной) и распаковал в три копии: `zk/server1`, `zk/server2`, `zk/server3`.



2. Ансамбль ZooKeeper

2.1 Конфигурация

В каждой копии свой `conf/zoo.cfg`: строки участников ансамбля (`server.N`) одинаковые, а порты и папка `dataDir` у каждой свои. Конфигурация первого сервера (`zookeeper/server1.cfg`):

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=./data
clientPort=2181
admin.enableServer=false
4lw.commands.whitelist=*
server.1=localhost:2888:3888
server.2=localhost:2889:3889
server.3=localhost:2890:3890
```

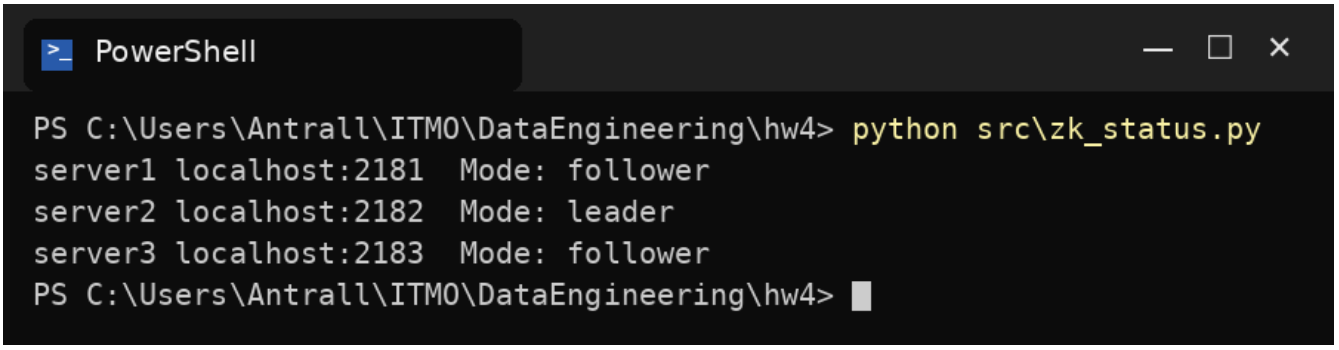
Сервер	Клиентский порт	Порты участника (обмен / выборы)	`dataDir`	`myid`
server1	2181	2888 / 3888	`zk/server1/data`	1
server2	2182	2889 / 3889	`zk/server2/data`	2
server3	2183	2890 / 3890	`zk/server3/data`	3

`dataDir=./data` указывает на папку внутри своей копии, и данные серверов не пересекаются. В `data/myid` лежит номер сервера: по нему сервер узнаёт, какая из строк `server.N` про него. Встроенный админ-сервер выключен (`admin.enableServer=false`). Команды из четырёх букв открыты (`4lw.commands.whitelist=*`), чтобы работала команда `srvr`, которую использует `zk_status.py`.

2.2 Запуск и роли

Запустил три сервера командой `zkServer.cmd`, предварительно перейдя в папку своей копии (`cd zk\server1`, затем `.\bin\zkServer.cmd`): путь `dataDir` в конфигурации относительный, и если запускать из другой папки, серверы начнут писать данные в одно место. Каждый сервер работает в своём окне консоли и пишет в него журнал (на скриншоте окно сервера 1: он ищет лидера и становится последователем). Роли просмотрены скриптом `src\zk_status.py`, который спрашивает у каждого сервера команду `srvr`:

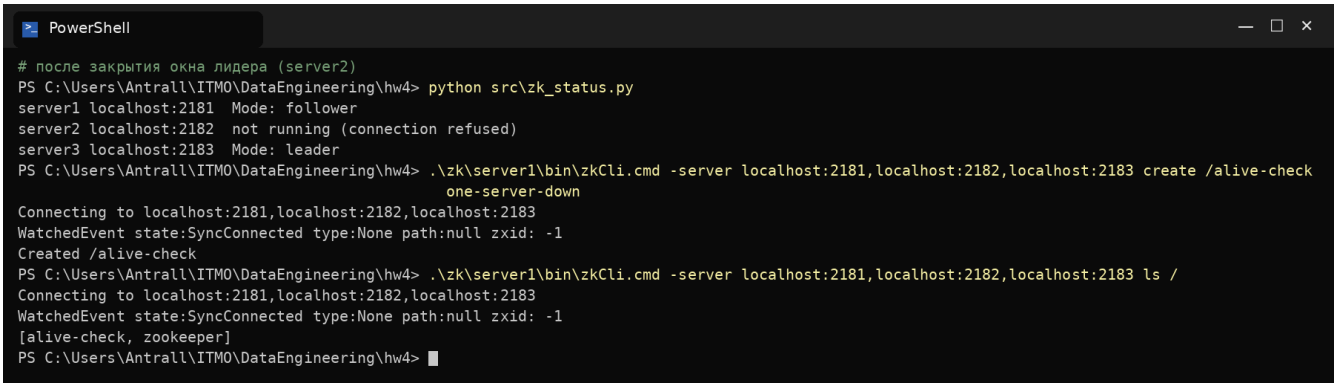




Лидер один (server2), два других следуют за ним (follower).

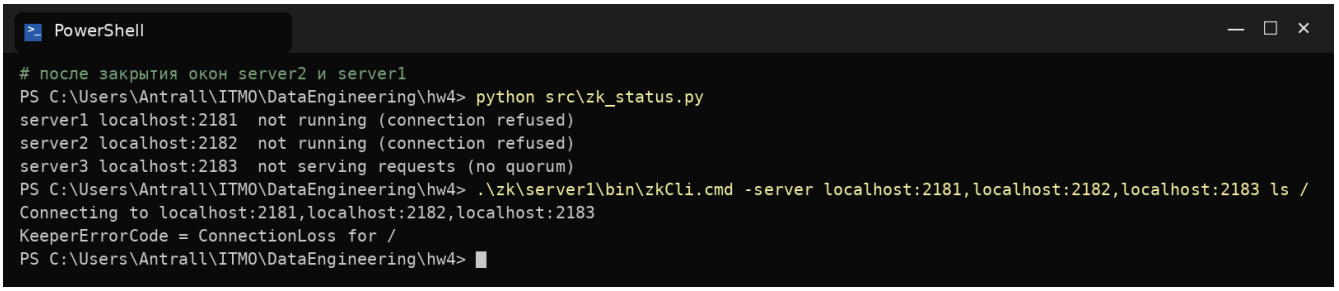
2.3 Потеря одного сервера

Закрыл окно лидера (server2), ну то есть остановил сервер. Оставшиеся два сервера провели выборы, новым лидером стал server3, а ансамбль продолжил работать: клиент создал узел `/alive-check` и прочитал список узлов.

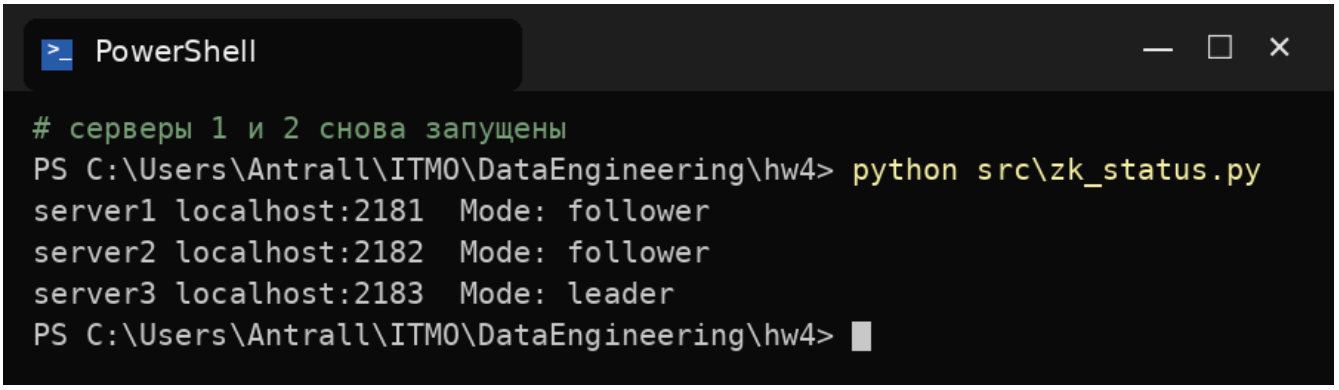


2.4 Потеря двух серверов

Закрыл еще окно второго сервера (server1). Остался один сервер из трёх, и кворума (2 из 3) нет: `zk_status.py` пишет, что server3 не обслуживает запросы, а клиент получает `ConnectionLoss` и ничего не может прочитать, хотя процесс server3 жив.

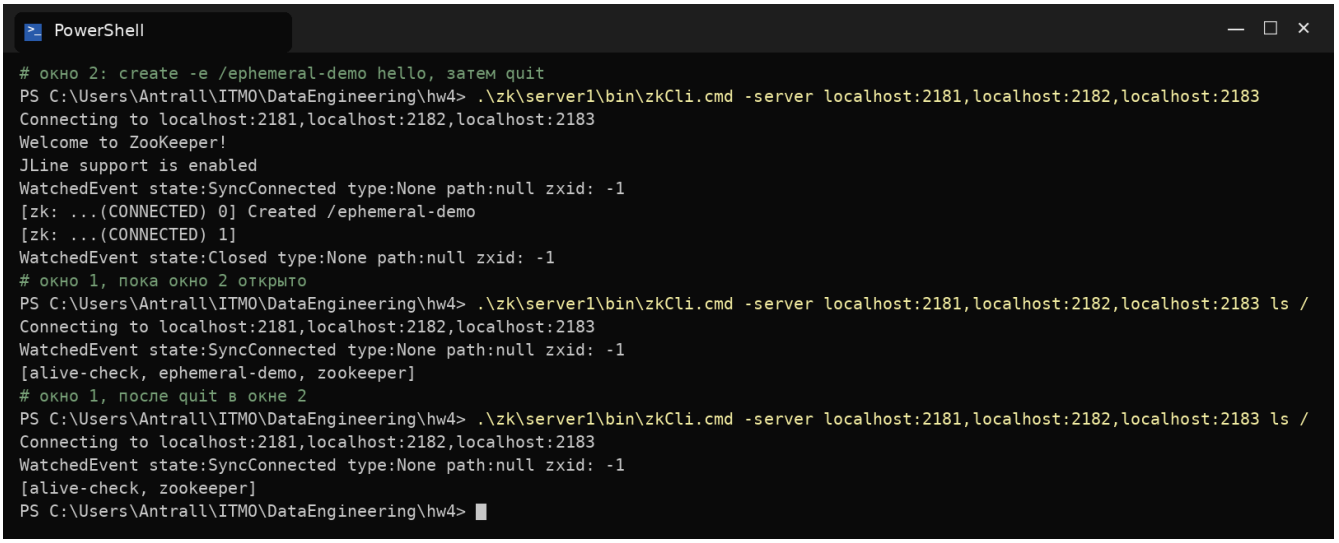


Потом запустил оба сервера снова (`zkServer.cmd` в двух окнах), ансамбль собрался (лидер по-прежнему server3):



2.5 Эфемерный узел

Во «втором» окне в `zkCli.cmd` создал эфемерный узел `/ephemeral-demo` (`create -e`) и держал сессию открытой. В «первом» окне узел виден в `ls /`. После закрытия второго окна (команда `quit` завершает сессию) узел исчез.



Это та же возможность, на которой построен выбор лидера ниже: узел привязан к сессии клиента, поэтому исчезает вместе с ней.

3. Координатор расчёта

Расчёт тот же, что в HW3: частоты **пар** навыков для вакансий варианта («Программист, разработчик на C и C++»), читает файл `raw/it_vacancies_2024.csv` (если что, там синтетика, но правдоподобная, сделал так в прошлых работах и уже оставил, поэтому некоторые числа могут не соотнестись с числами большинства) из хранилища чанками по 8 МиБ (34 чанка). Координатор запускается в двух копиях (A и B), код один и тот же.

Как устроена работа (`src/coordinator.py`):

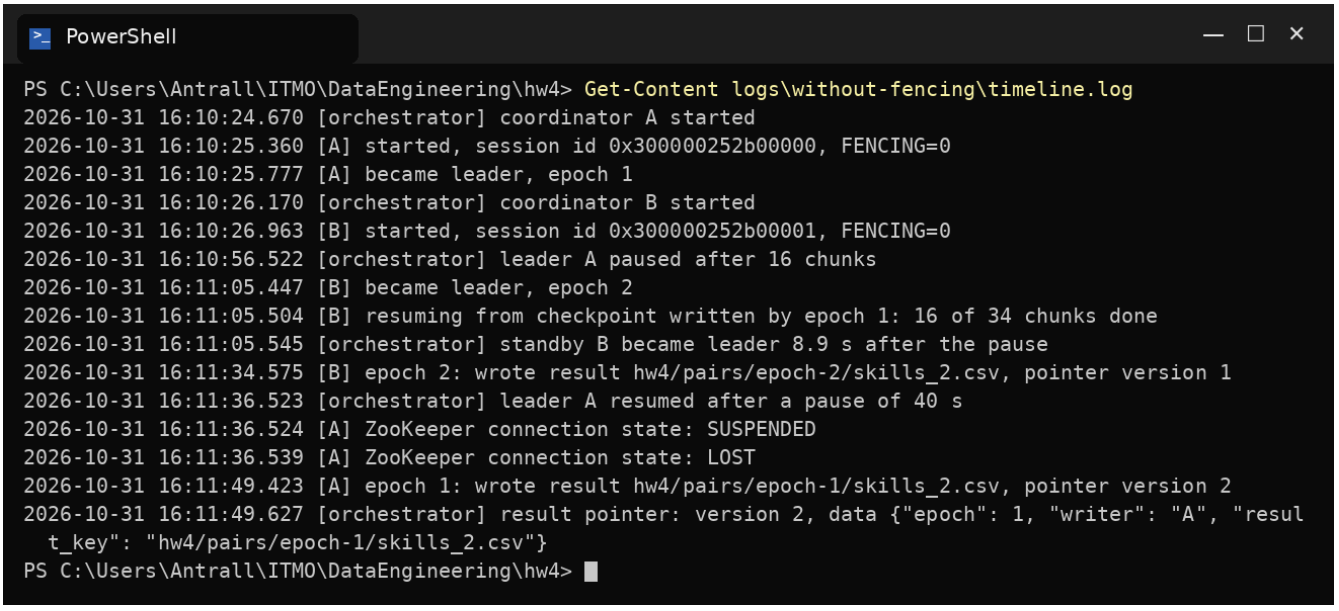
- **Выбор лидера.** Рецепт `Election` из `kazoo`: каждый координатор создаёт эфемерный последовательный узел в `/hw4/election`, лидер тот, у кого наименьший номер. Остальные ждут. Если сессия лидера закрыта, его узел исчезает, и следующий становится лидером.
- **Тайм-аут сессии 10 секунд.** Если ZooKeeper 10 секунд не получает от клиента сообщений, он закрывает сессию. Проверил по `cons`: согласованный тайм-аут сессии 10 000 мс.
- **Эпоха.** Каждый новый лидер увеличивает число в узле `/hw4/epoch` (с проверкой версии узла) и запоминает полученную эпоху и версию узла. Первый лидер получает эпоху 1, запасной после смены лидера эпоху 2.
- **Чекпоинт.** Лидер считает чанки по порядку и после каждого сохраняет в хранилище `hw4/pairs/checkpoint.json`: номера готовых чанков и накопленные счётчики пар. Новый лидер читает чекпоинт и продолжает с первого непосчитанного чанка.
- **Результат.** Готовый CSV кладётся в хранилище под ключом с номером эпохи (`hw4/pairs/epoch-N/skills_2.csv`), а затем обновляется узел-указатель `/hw4/result_pointer` (в нём эпоха, имя координатора и ключ результата). Версия узла растёт при каждой записи, по ней видно, сколько раз результат публиковали.
- **Задержка.** Весь расчёт на одном процессе занимает около 9 секунд (HW3), и лидера было бы не успеть остановить «посреди расчёта». Поэтому после каждого

чанка стоит искусственная пауза 0.6 с (параметр `--chunk-delay`), и расчёт идёт около 30-40 секунд.

Приостановку лидера делает `src/run_failover.py` через `psutil` (`Process.suspend()` замораживает процесс целиком): сначала запускается A, после его выбора лидером запускается B, после 16 готовых чанков A замораживается, примерно через 40 секунд после этого возобновляется (`resume()`).

4. Пауза лидера без защиты

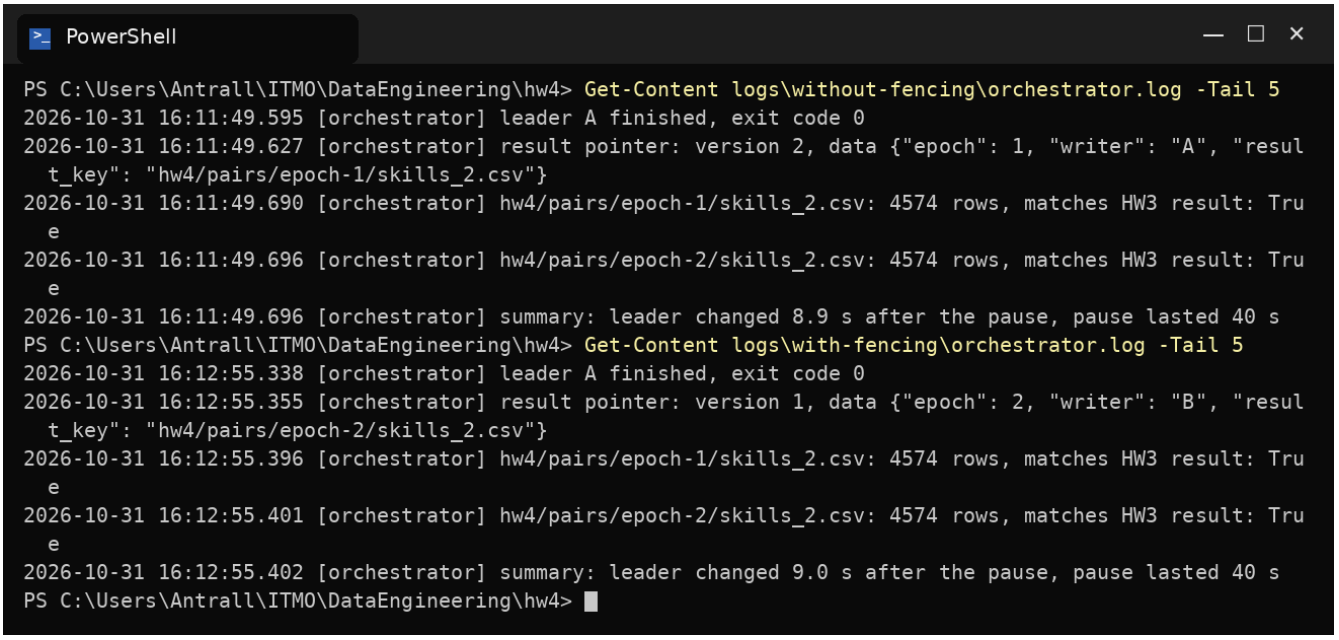
Запуск: `python src\run_failover.py --scenario without-fencing` (`FENCING` не задана). Хронология ключевых событий по журналам обоих координаторов и сценария (`logs/without-fencing/timeline.log`):



Что видно:

1. A стал лидером (эпоха 1) и посчитал 16 чанков из 34, после чего процесс A заморозили.
2. Через **8.9 с** после заморозки сессия A истекла (10 секунд без сообщений, а последнее сообщение было немного раньше заморозки), ZooKeeper удалил эфемерный узел A, и B стал лидером с эпохой 2.
3. B прочитал чекпоинт, записанный эпохой 1 («16 of 34 chunks done»), досчитал оставшиеся 18 чанков и записал результат: указатель получил версию 1.
4. Когда прошло 40 секунд паузы, A разморозили. Клиент ZooKeeper в A увидел, что сессия истекла (`SUSPENDED`, затем `LOST`; для следующих запросов он открыл новую сессию), но сам процесс ничего об этом не знал: он продолжил считать чанки по своему старому состоянию и по окончании записал результат **второй раз**: указатель получил версию **2** и теперь указывает на результат старой эпохи 1.

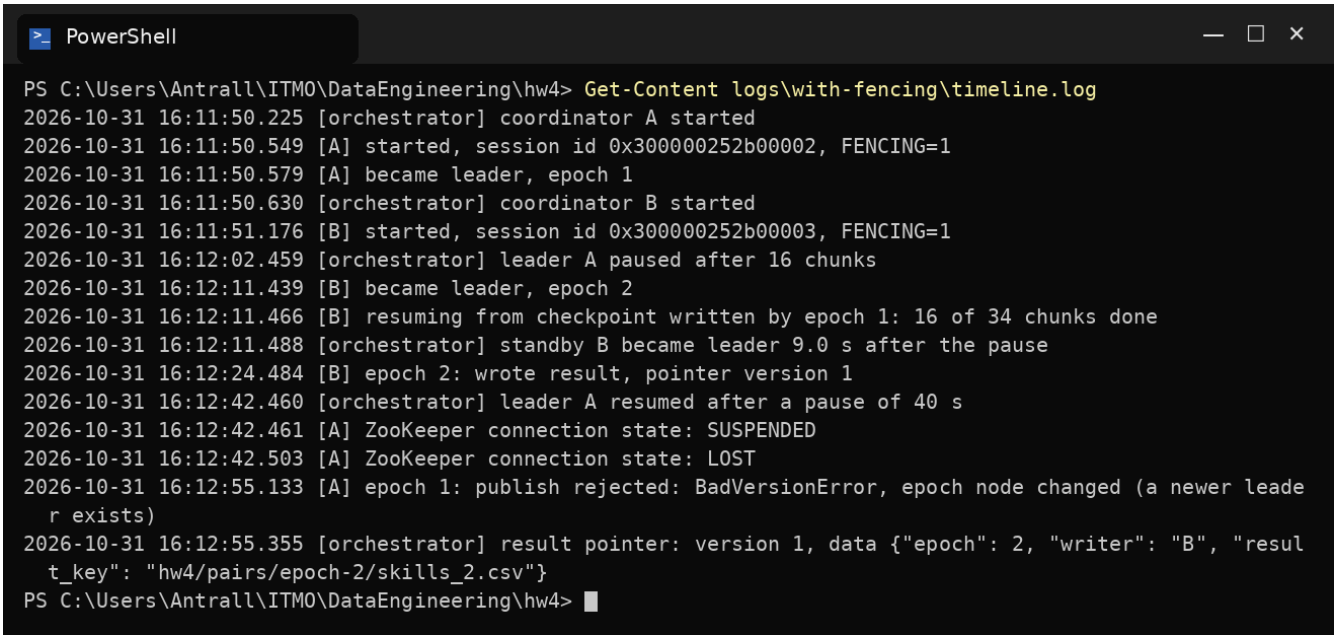
Итог запуска (`logs/without-fencing/orchestrator.log`):



Результаты обеих эпох совпали с результатом HW3 (4 574 пары, файл побайтово одинаковый), то есть B на самом деле доделал расчёт с чекпоинта верно. Но без защиты запись запоздавшего лидера отработала так же успешно, как запись настоящего: указатель оказался перезаписан результатом старой эпохи, и из лога видно две строки «wrote result». В этом запуске оба результата одинаковые, поэтому вреда не видно, но если бы старый лидер считал по устаревшим данным, он молча затёр бы правильный ответ.

5. Пауза лидера с защитой по эпохе

Запуск: `$env:FENCING = "1"; python src\run_failover.py --scenario with-fencing`. Всё то ж самое, но публикация результата теперь идёт транзакцией ZooKeeper из двух операций: проверка (`check`), что узел `/hw4/epoch` по-прежнему имеет ту версию, которую координатор получил, когда стал лидером, и только затем запись указателя. Если за это время появился новый лидер, версия узла эпохи изменилась, проверка не проходит и вся транзакция отклоняется.



Что видно: смена лидера произошла через **9.0 с** после заморозки, B доделал расчёт с чекпоинта (16 из 34 чанков уже были готовы) и записал указатель (версия 1, эпоха 2). Когда A разморозили (после 40 с паузы) и он дошёл до публикации, его запись получила `BadVersionError` и была отклонена. Указатель остался от запасного координатора: версия 1, эпоха 2, автор B. Результат старого лидера лежит в хранилище (`epoch-1/skills_2.csv`), но на него никто не ссылается; он совпадает с результатом HW3, но это уже не важно: в указатель попал только результат текущей эпохи.

Итого по двум запускам:

	Без защиты	С защитой по эпохе
Смена лидера после заморозки A	8.9 с	9.0 с
Длительность паузы A	40 с	40 с
Запись результата	дважды (B в эпоху 2, затем A в эпоху 1)	один раз (B); запись A отклонена: `BadVersionError`
Указатель `/hw4/result_pointer` в конце	версия 2, эпоха 1, автор A	версия 1, эпоха 2, автор B
Результат совпал с HW3	да (обе эпохи, 4 574 пары)	да (обе эпохи, 4 574 пары)

6. Ответы на вопросы

6.1 Почему серверов три, а не два и не четыре

ZooKeeper работает, пока живо **большинство** серверов (кворум): `N/2 + 1` с округлением вниз. Из этого:

Серверов	Кворум	Сколько отказов переживает
1	1	0
2	2	0

Серверов	Кворум	Сколько отказов переживает
3	2	1
4	3	1
5	3	2

- **Два сервера хуже одного.** Кворум равен 2, поэтому отказ любого из двух серверов останавливает ансамбль, и шансов на такой отказ больше, чем у одного сервера. Отказоустойчивости ноль, а сложность выше.
- **Четыре сервера не лучше трёх.** Кворум 3, поэтому всё равно переживается только один отказ, как у трёх. Зато на каждую запись нужно подтверждение от трёх серверов вместо двух (медленнее), больше сетевого обмена и больше серверов, которые могут сломаться. Кроме того, при разрыве сети 2+2 ни одна из половин не набирает кворум, и ансамбль остановится, тогда как у трёх серверов в любой разбивке одна из частей (2 из 3) кворум сохраняет.
- **Три сервера** это минимальная конфигурация, которая переживает отказ одного сервера. Это я проверил сам: без одного сервера ансамбль работал (п. 2.3), без двух нет (п. 2.4).

6.2 Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

ZooKeeper знает только про сессии: что клиент не присылал сообщений 10 секунд, и поэтому его сессия закрыта, а эфемерные узлы удалены. Но он не может остановить того, кто уже решил писать, и не может сообщить об этом замороженному процессу: тот в этот момент вообще ничего не делает. Замороженный лидер (пауза процесса, длительная сборка мусора, отключение сети) после пробуждения продолжает выполнять свой код с того же места и уверен, что он лидер.

Проверка «я лидер?» на стороне самого координатора ничего не даёт: она может пройти до заморозки, а запись случиться после (это ровно то, что произошло в п. 4: у A никаких проверок после пробуждения не было). Любая проверка, отделённая от записи, устаревает. Поэтому надёжно проверять эпоху можно только в точке, которая **принимает запись и сама решает, пустить её или нет**, и делать это атомарно вместе с записью. В моей реализации это узел-указатель: транзакция `check(/hw4/epoch, версия) + set(/hw4/result_pointer)` либо выполняется целиком, либо отклоняется целиком (`BadVersionError`). Токен (эпоха) приходит вместе с самой записью, а решение принимает тот, кто эту запись хранит. Это общий приём, он называется fencing token.

7. Выводы и ограничения

- Ансамбль из трёх серверов ZooKeeper 3.9.6 на JDK 17 собран; пока жив кворум (2 из 3), он работает, без кворума не отвечает. Эфемерный узел исчезает вместе с сессией.
- Выбор лидера на эфемерных узлах работает: при заморозке лидера запасной стал лидером через 8.9 и 9.0 с (тайм-аут сессии 10 с; время зависит от того, как давно лидер успел отправить последнее сообщение) и доделал расчёт с чекпоинта, результат совпал с HW3.

- Без защиты запоздавший лидер записал результат повторно (указатель версии 2 от старой эпохи); с защитой по эпохе его запись отклонена (`BadVersionError`), указатель остался за запасным координатором.